

AUTOdeal

Car Marketplace Web Application

Cortel Dennis-Alexandru
Group 30433/1

1 Problem Definition

The goal of this project is to develop a web application that allows users to browse, manage, and interact with car listings in an online marketplace environment.

Nowadays, many people use online platforms to buy or sell cars. These platforms need to be easy to use, secure, and flexible enough to handle different types of users. Because of this, the application must support multiple roles and allow users to perform different actions depending on their permissions.

The application developed in this project is called **AUTOdeal**. It is a car marketplace system where users can browse available cars, filter and sort listings, and for authenticated users, manage their own car posts.

The system supports three types of users:

- **Visitor** – can view and filter car listings
- **Seller** – can create, edit, and delete their own car posts
- **Admin** – has full control over all data

The application is built using **Spring Boot** and follows the **MVC (Model-View-Controller)** architecture. It uses **PostgreSQL** as the database and **Spring Security** for authentication and authorization.

The main goal is to provide a clean, structured, and secure system that satisfies the assignment requirements while also being realistic and usable.

2 Analysis Phase

2.1 System Overview

The system is a web-based application that allows users to interact with car listings. The main functionality revolves around managing and displaying car posts.

Each car post contains information such as:

- Brand and model
- Year
- Price
- Mileage
- Fuel type
- Horsepower
- City
- Visibility status

The application also includes user accounts, where each user has a role that determines what actions they can perform.

2.2 Functional Requirements

The main functional requirements of the system are:

- Users must be able to register and log in
- The system must support role-based access (Visitor, Seller, Admin)
- Users must be able to view car listings
- The system must allow filtering and sorting of car listings
- Sellers must be able to create new car posts
- Sellers must be able to edit and delete their own posts
- Admins must be able to manage all posts

2.3 Non-Functional Requirements

- The system should be easy to use and intuitive
- The application should be secure (password encryption, role control)
- The code should be modular and well-structured
- The system should be maintainable and scalable

2.4 Use Case Diagram

The use case diagram below presents the main actors of the system and the actions they can perform. A visitor can browse car listings, filter them, view details, and register or log in. A seller has all visitor capabilities and can also create posts and manage their own listings. The admin has the highest access level and can manage all application data.

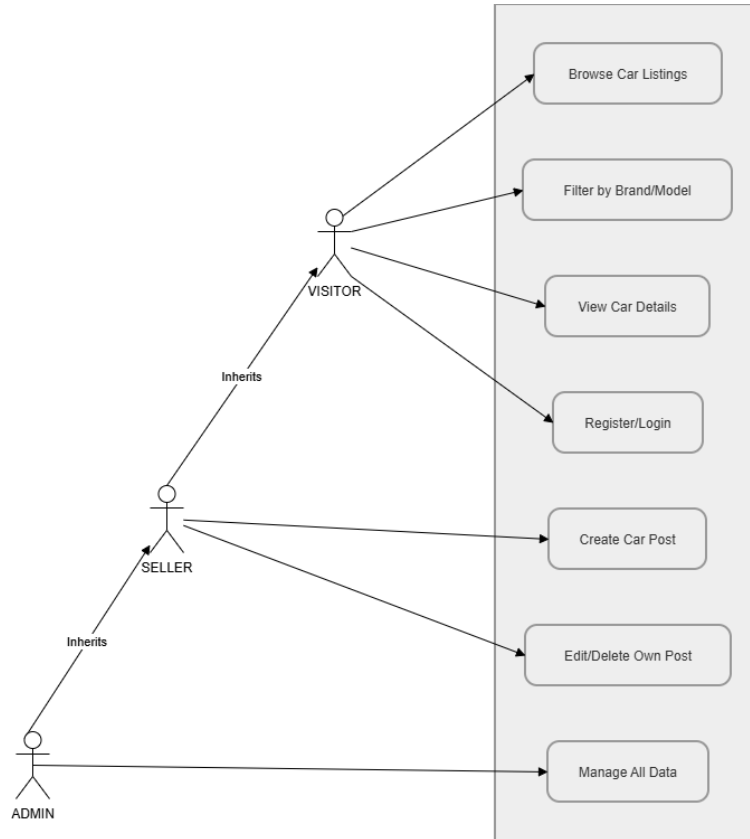


Figure 1: AUTOdeal Use Case Diagram

2.5 Use Case Descriptions

Browse Cars – A visitor or logged-in user can view all available car listings.

Filter Cars – Users can filter cars based on criteria such as brand, price, or fuel type.

Register/Login – Users can create accounts and authenticate into the system.

Create Post – A seller can add a new car listing.

Edit/Delete Post – A seller can modify or remove their own listings.

Admin Management – The admin can manage all users and posts.

3 Design Phase

3.1 Architecture

The application follows the **MVC (Model-View-Controller)** architecture.

- **Model** – represents the data (entities like User, CarPost, Brand)
- **View** – user interface implemented using Thymeleaf templates
- **Controller** – handles HTTP requests and connects the view with the business logic

This structure helps keep the code organized and easy to maintain. Each layer has a clear responsibility, which reduces complexity and improves scalability.

3.2 Layered Structure

The application is divided into multiple layers:

- **Controller Layer** – receives user requests and returns responses
- **Service Layer** – contains business logic
- **Repository Layer** – interacts with the database using JPA
- **Entity Layer** – defines the data model

This separation follows good design practices such as low coupling and high cohesion.

3.3 Database Design

The system uses a relational database (PostgreSQL).

Main entities:

- **User**
- **Brand**
- **CarModel**
- **CarPost**

Relationships:

- A CarPost belongs to one User (seller)
- A CarPost belongs to one Brand
- A CarPost belongs to one CarModel

3.4 ER Diagram

The database diagram below shows the structure of the PostgreSQL database used in the project. The `car_posts` table is the central table and references the seller, brand, and model through foreign keys. This design keeps the data normalized and makes relations between entities clear and easy to manage.

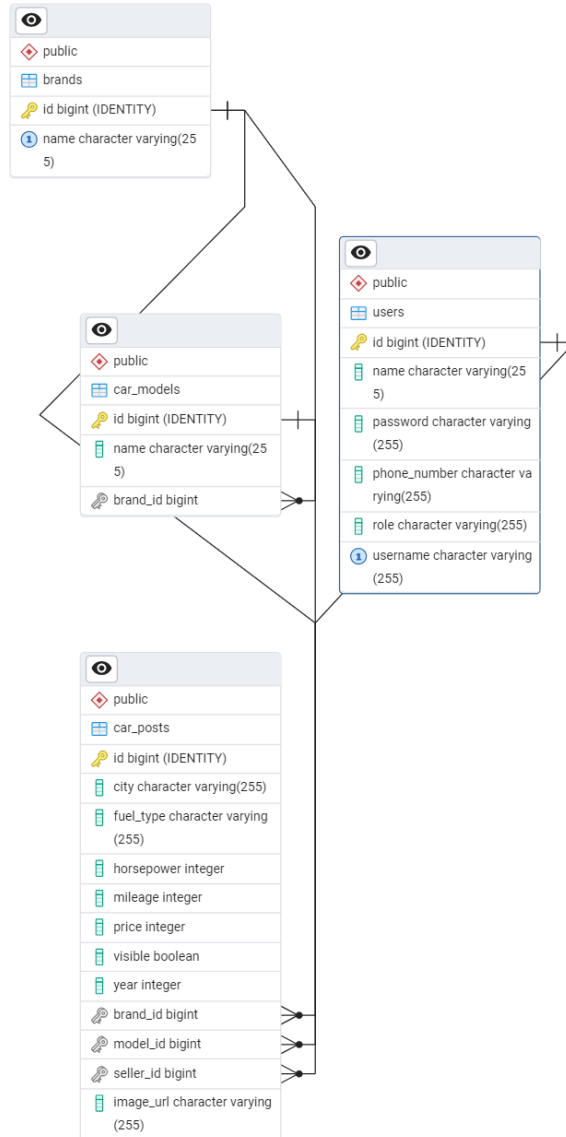


Figure 2: AUTodeal Database / ER Diagram

3.5 Class Diagram

The UML class diagram presents the main classes of the application and the relations between them. It includes the entities, repositories, services, controllers, security classes, and supporting components. This diagram reflects the layered architecture used in the project and shows how responsibilities are separated between components.

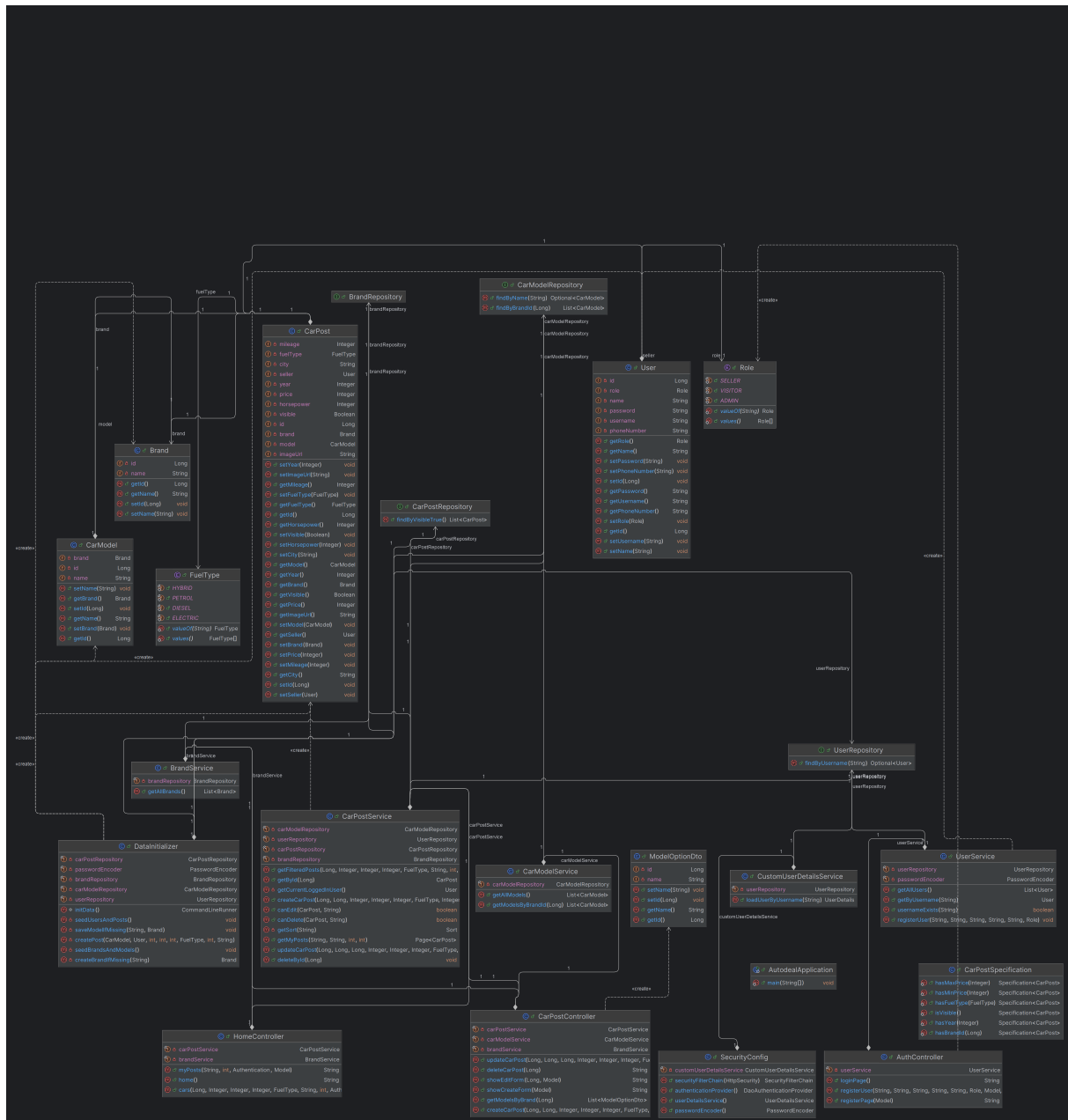


Figure 3: AUTOdeal UML Class Diagram

3.6 Sequence Diagram

The sequence diagram below illustrates the process of creating a new car post. The user fills in the form, the request is sent to the controller, then passed through the service and repository layers, and finally stored in the PostgreSQL database. After the operation is completed successfully, the user is redirected back to the cars page.

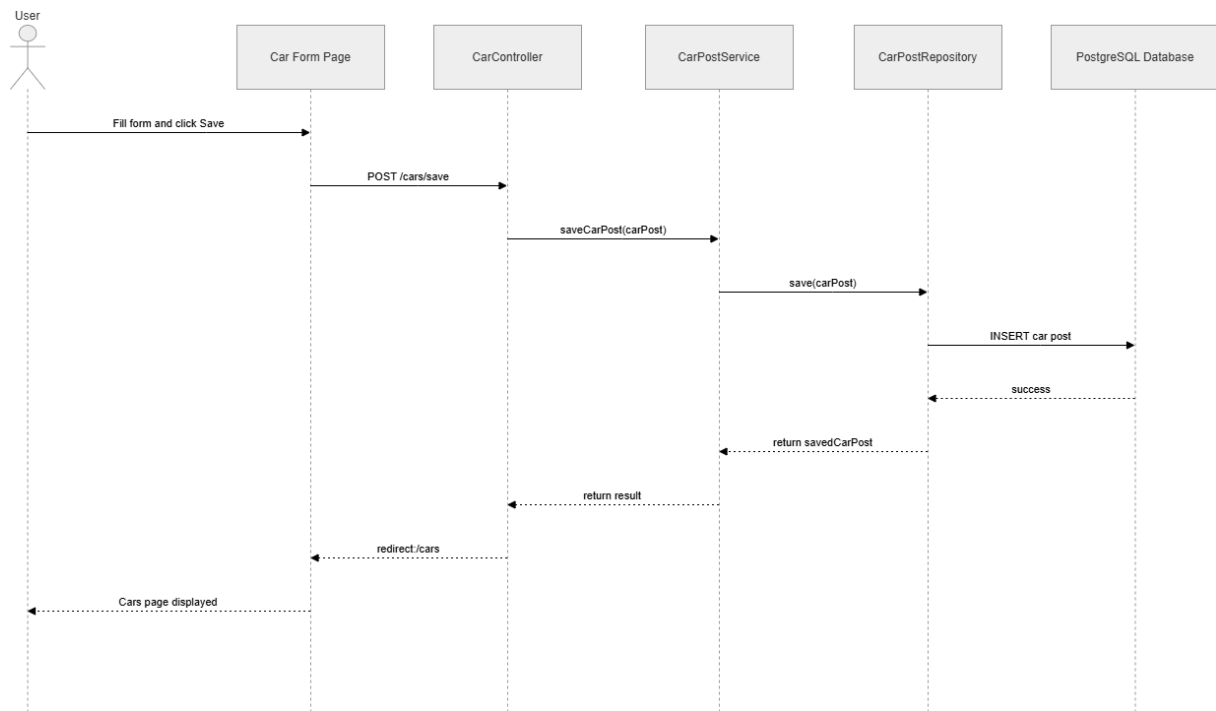


Figure 4: AUTOdeal Sequence Diagram for Creating a Car Post

4 Implementation Phase

4.1 Technologies Used

- Java + Spring Boot
- Spring MVC
- Spring Security
- Thymeleaf
- PostgreSQL
- JPA / Hibernate

4.2 Authentication and Security

Authentication is implemented using Spring Security.

Passwords are encrypted using **BCrypt**, which ensures that user credentials are stored securely.

Role-based access control is used to restrict access:

- Visitors – read-only access
- Sellers – can manage their own posts
- Admins – full access

4.3 Main Features

- Car listing display
- Filtering and sorting functionality
- Pagination for better performance
- CRUD operations for car posts
- User authentication and authorization

4.4 Project Structure

The project is organized into the following packages:

- `controller` – handles HTTP requests
- `service` – business logic
- `repository` – database access
- `model/entity` – data structures
- `templates` – Thymeleaf HTML pages

5 Testing and Validation

Testing was mainly done manually by running the application and checking all features.

The following scenarios were tested:

- User registration and login
- Role-based access control
- Creating, editing, and deleting car posts
- Filtering and sorting functionality
- Pagination behavior

Additionally, basic validation and logic checks were performed with the help of AI tools to ensure correctness and consistency.

The system behaved as expected in all tested scenarios.

6 Conclusion

The AUTOdeal application successfully implements a functional car marketplace using modern web technologies.

The project respects the MVC architecture, includes role-based access control, and provides important features such as filtering, sorting, and CRUD operations.

Overall, the application meets all assignment requirements and demonstrates a clear and structured approach to web development.